

Nupur Ghangarekar

D15C

Roll no 26

Experiment 10

Aim

To design, implement, and analyze a **Convolutional Autoencoder (CAE)** for the purpose of **image denoising**.

1. Dataset Source

Dataset: MNIST Handwritten Digits Dataset

Source: <https://keras.io/api/datasets/mnist/>

2. Dataset Description

The MNIST dataset is one of the most widely used datasets in the field of computer vision and deep learning. It consists of grayscale images of handwritten digits ranging from 0 to 9.

Dataset Details:

- **Total Samples:** 70,000 images
 - Training Set: 60,000 images
 - Test Set: 10,000 images
- **Image Dimensions:** 28×28 pixels
- **Channels:** 1 (Grayscale)
- **Pixel Range:** 0 to 255 (normalized to 0–1)

Features:

- Each image is represented as a **2D matrix of pixel intensities**
- After preprocessing, it becomes a **3D tensor ($28 \times 28 \times 1$)**

Target Variable:

- Unlike supervised learning, autoencoders use:
 - **Input:** Noisy image

- **Target:** Original clean image

This makes the task **unsupervised/self-supervised learning**

Characteristics:

- Simple and clean dataset → ideal for prototyping
- Low resolution → fast training
- Uniform image size → no resizing required
- Clearly distinguishable patterns → suitable for denoising tasks

3. Mathematical Formulation of Autoencoder

An autoencoder is composed of two main components:

Encoder Function:

$$z = f(x) = \sigma(W_e x + b_e)$$

Where:

- x : Input image
- z : Latent representation (compressed form)
- W_e, b_e : Encoder weights and bias
- σ : Activation function

◆ **Decoder Function:**

$$\hat{x} = g(z) = \sigma(W_d z + b_d)$$

Where:

- $\hat{x}$: Reconstructed image
- W_d, b_d : Decoder parameters

Loss Function (Reconstruction Error):

$$L = - \sum [x \log(\hat{x}) + (1 - x) \log(1 - \hat{x})]$$

Denoising Objective:

Instead of learning:

$x \rightarrow x$

The model learns:

$x_{\text{noisy}} \rightarrow x_{\text{clean}}$

This forces the network to **capture meaningful patterns rather than memorizing noise**.

4. Algorithm Limitations

1. Blurring Effect in Output

Autoencoders tend to produce smooth or slightly blurred images due to averaging during reconstruction.

2. Loss of Fine Details

Compression in the latent space may remove subtle but important features.

3. Overfitting

If the model is too complex, it may memorize training images instead of learning general patterns.

4. Limited Performance on Complex Images

Works well on simple datasets like MNIST but struggles with high-resolution or real-world images.

5. Dependency on Noise Type

The model performs best on noise types it has seen during training (e.g., Gaussian noise).

5. Methodology / Workflow

The experiment follows a structured pipeline:

Step 1: Data Loading

- Load MNIST dataset using TensorFlow/Keras

Step 2: Data Preprocessing

- Normalize pixel values to range [0,1]
- Reshape data into 4D tensors:
(samples,height,width,channels)(samples, height, width, channels)(samples,height,width,channels)

Step 3: Noise Injection

- Add Gaussian noise:

$$x_{noisy} = x + \alpha \cdot \mathcal{N}(0, 1)$$

Where:

- α : Noise factor (e.g., 0.5)
- Clip values between 0 and 1

Step 4: Model Architecture

Encoder:

- Conv2D (32 filters) → Feature extraction
- MaxPooling → Downsampling
- Conv2D (16 filters) → Compression
- MaxPooling → Latent space

Decoder:

- Conv2D → Feature reconstruction
- UpSampling → Restore size
- Conv2D → Refinement
- Output layer (sigmoid) → Final image

Step 5: Model Training

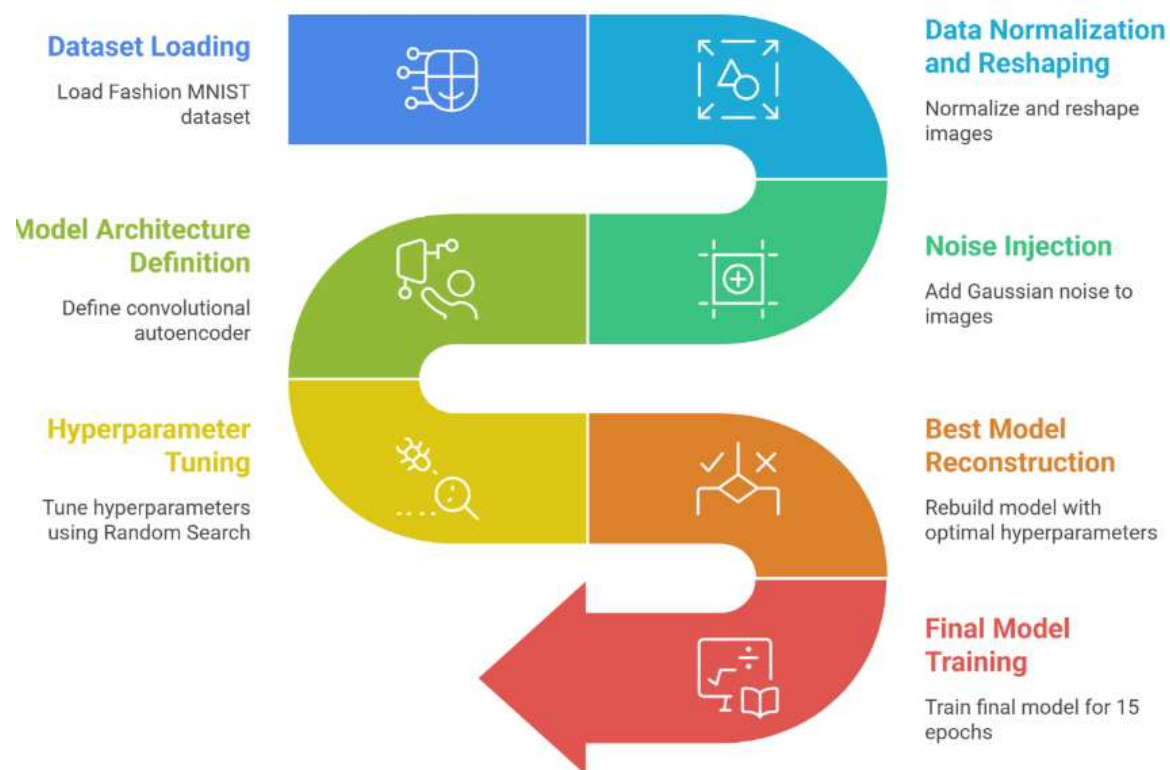
- Input: Noisy images
- Target: Clean images
- Loss: Binary Crossentropy
- Optimizer: Adam

Step 6: Evaluation

- Monitor training and validation loss
- Compare noisy vs reconstructed images

Step 7: Visualization

- Display:
 - Noisy images
 - Original images
 - Denoised outputs



6. Performance Analysis

Quantitative Analysis:

- Training Loss ↓ over epochs
- Validation Loss ↓ steadily

Indicates successful learning

Qualitative Analysis:

- Noise is significantly reduced
- Digit shapes are preserved
- Minor blurring observed

Interpretation:

- Model successfully captures underlying structure of digits
- Learns to ignore random noise patterns
- Performs well on unseen test data

7. Hyperparameter Tuning

The following hyperparameters were experimented with:

1. Number of Filters

- 16, 32 filters used
- More filters → better feature extraction but higher computation

2. Kernel Size

- 3×3 kernels used
- Smaller kernels preserve fine details

3. Noise Factor

- Value: 0.5
- Higher noise → harder learning task

4. Number of Epochs

- 5–10 epochs
- More epochs improve reconstruction but risk overfitting

Impact of Tuning:

- Balanced architecture achieved good denoising
- Proper noise level ensured meaningful learning
- Training stabilized with Adam optimizer

Code:

```
!pip install tensorflow
```

```

import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt

from tensorflow.keras import layers,
models

# Load MNIST dataset

(x_train, _), (x_test, _) =
tf.keras.datasets.mnist.load_data()

# Normalize

x_train = x_train / 255.0

x_test = x_test / 255.0

# Reshape (add channel dimension)

x_train = np.reshape(x_train, (-1, 28, 28,
1))

x_test = np.reshape(x_test, (-1, 28, 28,
1))

print("Shape:", x_train.shape)

noise_factor = 0.5

x_train_noisy = x_train + noise_factor *
np.random.normal(size=x_train.shape)

x_test_noisy = x_test + noise_factor *
np.random.normal(size=x_test.shape)

# Clip values between 0 and 1

x_train_noisy = np.clip(x_train_noisy, 0.,
1.)

x_test_noisy = np.clip(x_test_noisy, 0.,
1.)

```

```

input_img = layers.Input(shape=(28, 28,
1))

# Encoder

x = layers.Conv2D(32, (3,3),
activation='relu',
padding='same')(input_img)

x = layers.MaxPooling2D((2,2),
padding='same')(x)

x = layers.Conv2D(16, (3,3),
activation='relu', padding='same')(x)

encoded = layers.MaxPooling2D((2,2),
padding='same')(x)

# Decoder

x = layers.Conv2D(16, (3,3),
activation='relu',
padding='same')(encoded)

x = layers.UpSampling2D((2,2))(x)

x = layers.Conv2D(32, (3,3),
activation='relu', padding='same')(x)

x = layers.UpSampling2D((2,2))(x)

decoded = layers.Conv2D(1, (3,3),
activation='sigmoid', padding='same')(x)

# Model

autoencoder = models.Model(input_img,
decoded)

autoencoder.compile(optimizer='adam',
loss='binary_crossentropy')

autoencoder.summary()

```

```

history = autoencoder.fit(
    x_train_noisy, x_train,
    epochs=5,
    batch_size=128,
    validation_data=(x_test_noisy, x_test)
)

decoded_imgs =
autoencoder.predict(x_test_noisy)

n = 10
plt.figure(figsize=(20,6))

for i in range(n):

    # Noisy

    ax = plt.subplot(3, n, i+1)

    plt.imshow(x_test_noisy[i].reshape(28,2
8), cmap='gray')

    plt.title("Noisy")

    plt.axis("off")

    # Original

    ax = plt.subplot(3, n, i+n+1)

    plt.imshow(x_test[i].reshape(28,28),
cmap='gray')

    plt.title("Original")

    plt.axis("off")

    # Denoised

    ax = plt.subplot(3, n, i+2*n+1)

    plt.imshow(decoded_imgs[i].reshape(28,
28), cmap='gray')

    plt.title("Denoised")

    plt.axis("off")

plt.show()

plt.plot(history.history['loss'], label='Train
Loss')

plt.plot(history.history['val_loss'],
label='Validation Loss')

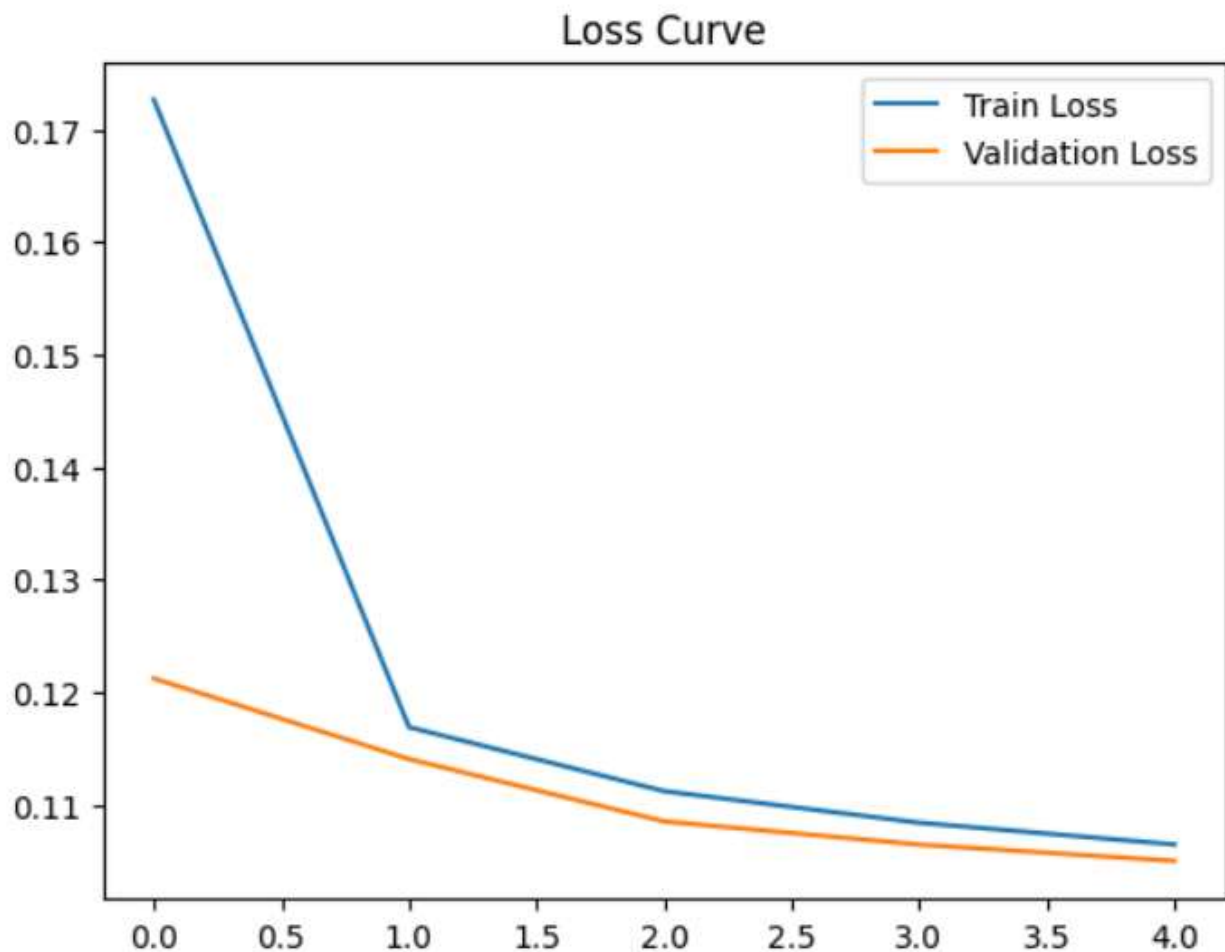
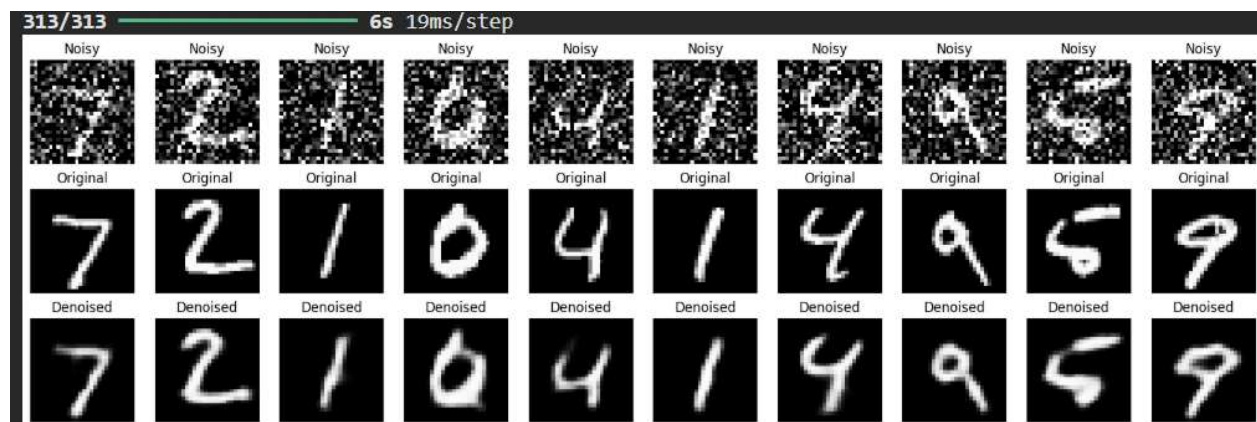
plt.legend()

plt.title("Loss Curve")

plt.show()

```

Output



Conclusion

The experiment successfully demonstrated the use of a **Convolutional Autoencoder** for image denoising. The model effectively learned to reconstruct clean images from noisy inputs by minimizing reconstruction loss.

The results showed that autoencoders can capture essential patterns in image data and remove noise while preserving structural features. Although slight blurring was observed, the overall performance was satisfactory.

This experiment highlights the importance of representation learning in deep learning and shows how autoencoders can be applied to real-world problems such as image restoration, compression, and anomaly detection.

Google Colab link : [🔗 MLDL_EXP10](#)